Project Progress Report

Unsupervised Learning Project | Milestone 2

Team	Francisco Rodrigues dos Santos (72908) · Duarte Ferreira (75419) · João Rivera (73418)
Project title	Hotel Booking Demand: Clustering of Booking Behaviour Profiles
Checkpoint date	30 April 2026
Repository	*Github*

1. RQ1 and segmentation time

Segmentation question

What recurring booking-behaviour profiles — defined over stay length, seasonality, distribution channel and commitment level — emerge from hotel reservation records, using only information available at the moment of booking confirmation? The goal is an exploratory segmentation usable for operational and marketing decisions at booking time. Clustering is justified because no supervised label captures "behaviour at booking time".

Unit of analysis

One row equals one booking record (not a customer). Conclusions are framed as recurring booking behaviours, not customer types.

Segmentation (index) time

Booking confirmation. Every clustering input is available at or before that point. Anything updated afterwards (cancellation outcome, room re-assignment, waiting-list days, post-event status) is reserved for post-hoc profiling only.

2. Dataset version control and documentation

Course release v1 (`hotel_bookings_course_release_v1.csv`), 119,390 rows by 32 columns. SHA-256 fingerprint `7c2ae42a...c7b1fc06`, recorded in `data/raw/DATASET_MANIFEST.yml`. Source: Kaggle (`jessemostipak/hotel-booking-demand`, CC BY 4.0); peer-reviewed reference António, de Almeida and Nunes (2019), *Data in Brief*, doi:10.1016/j.dib.2018.11.126. Raw CSV is not committed.

Known data-quality issues

- `company` missing in 94.3 % of rows and `agent` in 13.7 %; both are ID-like and dropped.
- `country` missing in 0.41 % and high-cardinality (~93 codes); rare-grouped at minimum frequency 100.
- `meal == "Undefined"` and a few negative `adr` values are recoded to NaN before imputation.
- Heavy right-skew in `lead_time`, `previous_cancellations`, `previous_bookings_not_canceled`, which is why we keep RobustScaler as a co-equal scaler variant.
- The four raw `arrival_date_*` columns are flagged by the course's `column_roles.csv` for cyclic encoding.

3. Completed Exploratory Data Analysis (EDA)

Missingness summary

Column	Type	n_missing	%
company	categorical	112,593	94.31
agent	numerical (ID)	16,340	13.69
country	categorical	488	0.41
children	numerical	4	0.003
All other 28 columns	—	0	0.00

Outlier summary (1.5·IQR fences, numerical clustering inputs)

lead_time (Q1=18, Q3=164, fence 383), stays_in_week_nights (Q1=1, Q3=3, fence 6) and stays_in_weekend_nights (Q1=0, Q3=2, fence 5) are kept: long lead-times and long stays are real booking behaviour. The remaining numerical inputs (adults, children, babies, previous_cancellations, previous_bookings_not_canceled) all have zero IQR — non-zero values are exactly the signal we want, so rows are not removed. Heavy tails are absorbed by median imputation plus Standard or Robust scaling.

Variables excluded from clustering inputs

- **Mandatory leakage exclusions** (post-event): is_canceled, reservation_status, reservation_status_date.
- **Post-confirmation exclusions**: assigned_room_type, booking_changes, days_in_waiting_list.
- **Identifier exclusions**: agent, company.
- **Re-coded into cyclic pairs and then dropped from the clustering input**: arrival_date_month becomes (arrival_month_sin, arrival_month_cos); arrival_date_week_number becomes (arrival_week_sin, arrival_week_cos).
- **Re-coded into engineered booking-shape features**: stays_in_weekend_nights, stays_in_week_nights, adults, children, babies are collapsed into total_nights, party_size, has_kids, weekend_share so the clustering distance sees behavioural shape rather than five correlated count axes.
- **Dropped outright**: arrival_date_year (3-value dataset-window artefact, Jul-2015 to Aug-2017) and arrival_date_day_of_month (sub-monthly noise).
- **Profiling-only** (kept in the profiling frame, never enter the distance computation): meal, required_car_parking_spaces, total_of_special_requests, adr, is_repeated_guest, previous_cancellations, previous_bookings_not_canceled, country. Price is a downstream consequence of the booking choices we cluster on; the three guest-history fields encode customer identity rather than booking behaviour, so they are kept for profiling but excluded from inputs to keep the segmentation focused on reservations; country is treated as a control variable to avoid using nationality as a proxy for cluster identity.

All exclusions are centralised in src/preprocessing/feature_config.py.

4. Main representation (RQ2)

Included variables

- **Numerical (7)**: `lead_time`, the four engineered booking-shape features `total_nights`, `party_size`, `has_kids`, `weekend_share`, and the two cyclic week-of-year features `arrival_week_sin`, `arrival_week_cos`. The week-of-year cycle already wraps December-into-January, so a separate month cycle would only duplicate the seasonal signal and double-count it in the Euclidean distance.
- **Categorical (6)**: `hotel`, `market_segment`, `distribution_channel`, `reserved_room_type`, `deposit_type`, `customer_type`.

Excluded and profiling-only variables

Leakage and identifier exclusions are listed in section 3. Specific to the representation, eight variables are kept in the profiling frame but never enter the distance: `meal`, `required_car_parking_spaces`, `total_of_special_requests`, `adr`, `is_repeated_guest`, `previous_cancellations`, `previous_bookings_not_canceled`, `country`. Each is observable at booking time, so none is a leakage case. The first four are *consequences* of the booking choices we cluster on (price, on-site requests); the three guest-history fields encode customer identity rather than booking behaviour, which would pull the segmentation toward returning-guest signals instead of reservation shapes; `country` is held out as a control variable to avoid letting nationality act as a proxy for cluster identity. Including any of them would conflate cluster *definition* with cluster *description*.

Numerical preprocessing

Median imputation, then scaling. Cyclic encoding for both month and week-of-year so December and January (or week 52 and week 1) are close in feature space rather than 11 or 51 units apart. Two scaler variants run side-by-side under the same protocol: **StandardScaler** (zero mean, unit variance) and **RobustScaler** (median, IQR-based, resistant to long tails). We report both so any conclusion has to hold up under either.

Categorical preprocessing

Imputation with constant value `"Unknown"`. A custom `RareCategoryGrouper` collapses categories below 50 occurrences into `"Other"` before encoding. `OneHotEncoder` then maps unseen categories at scoring time to all-zeros, in a single encoding pass without double-counting. A `VarianceThreshold` step (prevalence floor 0.5 %) drops one-hot dummies that are too rare to define a stable distance dimension, and the surviving dummies are passed through `StandardScaler` so each lives on the same scale as the numerical block.

Block balancing

After encoding, the categorical block has many more columns than the numerical block (23 vs 7 on the 5,000-row fast-check). A `BlockWeighter` step rescales the categorical block by `sqrt(n_num / n_cat)` so the two blocks contribute comparable variance to the Euclidean distance instead of letting the wider block dominate by mere column count.

Final representation

The clustering matrix is dense, **Euclidean**, with 7 numerical plus 23 surviving one-hot dimensions, totalling **30 features on the 5,000-row fast-check** (block weight 0.552). Internal indices in section 5 are computed in this same space, with no metric switch in between.

5. Baseline modelling evidence

Methods and configuration

- **Method 1**: MiniBatch K-Means (`n_init=10`, `batch_size=1024`, `max_iter=300`). The centroid update rule and Euclidean objective are identical to full-batch K-Means; only the per-step subsampling differs. Mini-batch is needed at n = 119k for a tractable seed sweep.
- **Method 2**: iK-means (Mirkin, 2005); auto-determines K via anomalous-pattern peeling on a PCA-projected discovery space, then refits in the full 30-dim space.
- **Predefined K range**: {2, 3, 4, 5, 6, 7, 8}; bounded a-priori (below 2 trivial, above 8 not interpretable as an RQ1-actionable segmentation for operational and marketing decisions at booking time).
- **Seeds**: ten lock-stepped seeds {0..9}, giving 45 ARI pairs per (method, scaler, K) cell.
- **Distance**: Euclidean. **Silhouette** is sub-sampled (2,000 / 5,000 points per seed) to keep its $O(n^2)$ cost tractable.

Internal indices (5,000-row fast-check, *p* = 30, mean ± std over 10 seeds)

K	Sil. (Std)	Sil. (Rob)	DB (Std)	DB (Rob)	CH (Std)	CH (Rob)
2	0.1359 ± 0.1074	**0.1674 ± 0.0458**	2.75	2.15	10396	10901
3	0.0977 ± 0.0343	0.1375 ± 0.0196	2.70	2.33	9858	11332
4	0.0925 ± 0.0151	0.1521 ± 0.0190	2.55	2.03	9813	11447
5	0.0945 ± 0.0126	0.1580 ± 0.0161	2.35	2.01	9509	11670
6	0.1023 ± 0.0101	0.1512 ± 0.0224	2.26	1.96	9634	11283
7	**0.1111 ± 0.0151**	0.1652 ± 0.0189	2.19	1.85	9210	10921
8	0.1020 ± 0.0152	0.1646 ± 0.0187	2.08	1.86	8852	10561

DB and CH cells show the seed-mean only; per-seed std is logged in `experiments.csv` and is omitted here for table compactness.

Under Robust the silhouette landscape is flat from K = 2 to K = 8 (all in the 0.15–0.17 band), with K = 2 slightly best (0.1674 ± 0.0458). **Under Standard, K = 2 has the highest mean silhouette (0.1359) but is seed-fragile** (std 0.1074 ≈ mean), so K = 7 (0.1111 ± 0.0151) is the most stable silhouette-informative K. DB and CH favour higher K under both scalers — as expected, they reward smaller, denser clusters.

iK-means (auto-K) and runtime

iK-means produced two perfectly stable partitions (mean pairwise ARI = 1.0 across all 10 seeds): **K = 2 under RobustScaler with silhouette 0.8491**, and K = 2 under StandardScaler with silhouette 0.3316. The high robust-scaler silhouette reflects iK-means' anomalous-pattern peeling working as designed on long-tailed inputs: each seed reaches the same partition because the peeling order is deterministic given the data. iK-means-Robust K = 2 is the strongest baseline candidate at this checkpoint.

The fast-check pipeline (`bash run_all --fast`, Windows 11, Python 3.13) runs k-means and iK-means end-to-end in about 2 minutes (140 k-means fits in ~90 s, 20 iK-means fits in ~15 s). The full 119,390-row pass is part of the 13-May Core-Protocol-Freeze.

First cluster-interpretation note (iK-means, RobustScaler, K = 2)

The two iK-means clusters split along stay-length, distribution channel and deposit commitment, the three RQ1-actionable axes. The post-hoc profiling layer (built from the profiling frame, with `is_canceled` held out of clustering) shows non-trivial differences between the two groups on lead-time, total nights, online-TA share, deposit type, and cancellation rate. That is a first sign the partition is picking up real behavioural variance and not just noise; full numerical profiles for both partitions are in `tables/task1_2_summary.csv` and `figures/task1_2_*.png`.

6. Sensitivity / robustness plan

Main representation: the canonical clustering matrix defined in section 4 (7 numerical + 23 one-hot dimensions on the fast-check, block-balanced, Euclidean). The variants below are perturbations of this matrix.

Resampling rule: 10 lock-stepped seeds `{0..9}` per (method, scaler, K) cell, giving 45 ARI pairs per cell. Same seed scheme as section 5.

Stability evidence so far

Stability is measured as **mean pairwise Adjusted Rand Index** over 45 seed pairs at fixed (method, scaler, K). Selection rule: ARI of at least 0.80 to accept a K as stable.

Method	K	ARI (StandardScaler)	ARI (RobustScaler)
MiniBatch K-Means	2	0.149	0.128
MiniBatch K-Means	5	0.317	0.551
MiniBatch K-Means	6	0.307	0.539
MiniBatch K-Means	7	0.349	0.607
MiniBatch K-Means	8	**0.359**	**0.633**
iK-means	2	**1.000**	**1.000**

MiniBatch K-Means is silhouette-informative at K = 2 but seed-fragile: no MiniBatch K reaches the 0.80 threshold under either scaler, although Robust ARI now climbs above 0.6 from K = 7 onwards. iK-means, by contrast, hits perfect stability at K = 2 under both scalers, which is what makes iK-means-Robust K = 2 the strongest baseline candidate.

Controlled variants planned

Two comparisons, anchored against the canonical representation under StandardScaler:

1. **Standard vs Robust scaler** — already running in the same pipeline; the table above quantifies it. Does the silhouette-best K and its stability survive a scaler swap from Standard to Robust? At this checkpoint Robust strengthens both the K = 2 silhouette (0.1674 vs 0.1359) and the K = 8 ARI (0.633 vs 0.359); iK-means consistently selects K = 2 under both scalers.
2. **With vs without PCA-20** — does compressing the 30-dim space to 20 PCs change which K is silhouette-best? Particularly relevant for iK-means, whose discovery phase already runs in PCA space, so a controlled PCA pre-step lets us check whether iK-means' K choice is driven by the discovery dimensionality or by the data geometry itself.

Acceptance bar. A partition is considered "robust" only if it (i) maximises silhouette in at least 2 of the 3 variants (canonical-Standard, canonical-Robust, canonical-PCA20) and (ii) reaches mean ARI of at least 0.80 in at least one variant. iK-means already meets the ARI condition under both scalers; the open question is whether it survives the PCA-20 perturbation.

7. Experiment logging and repository status

`experiments.csv` is auto-appended at every run via `src/utils/experiment_logger.py` (160 rows after the latest committed run: 7 K × 2 scalers × 10 seeds for MiniBatch K-Means, plus 2 scalers × 10 seeds for iK-means). Schema: `task, method, variant, k, seed, silhouette, calinski_harabasz, davies_bouldin, ari_vs_seed0, bic, aic, log_likelihood, n_iter, converged, notes`. A `date / run_id` pair, a `parameters` JSON column (full hyperparameter vector), and a `sample_rule` column (`fast-check-5k` vs `full-119k`) will be added before the 13-May Core-Protocol-Freeze.

`environment.yml` (conda, Python 3.11) is checked in; a pinned `requirements.txt` for the actual lab environment (Python 3.13 on Windows 11) is being prepared from `pip freeze` for the 20-May Reproducibility Freeze.

`run_all` (bash) executes the Task 1 pipeline end-to-end: SHA-256 validation, preprocessing, baseline clustering with k-means and iK-means under both scalers. Every figure in `figures/` and every table in `tables/` is regenerated without manual steps.